PROJECT REPORT · OCTOBER 2026

Meet Translate

Video calls where a Hindi, Telugu or English speaker is heard by the other person in **their own language**, live, with captions.

3	6 / 6	3
languages: Hindi, Telugu, English	translation directions working	AI models compared (Google + OpenAI)

Live site: meet-translate.vercel.app
Code: github.com/mujeebmasi/meet-translate
Built: 12–14 October 2026 · Next.js, NestJS, WebRTC, Google Gemini, OpenAI

What's inside

1. The project in one page
2. System design (with diagrams)
3. The AI models we used (in order), and why Gemini won
4. How we built it, step by step
5. Problems we faced and how we solved them
6. Results, tech stack, limits and next steps
7. Glossary of the technical words

1. The project in one page

The problem

Two people want to talk on a video call, but they don't share a language. Ravi speaks Hindi, his colleague speaks Telugu, a client speaks English. Normally they'd need a human interpreter.

What we built

A website for one-to-one video calls with a built-in AI interpreter. Each person picks the language they speak. When Ravi talks in Hindi, the other person **hears a translated voice** in their own language instead of Ravi's voice, and sees **captions** of both the original and the translation.

A real example from testing

Speaker says (Hindi): आप कैसे हैं? चलिए आज की मीटिंग शुरू करते हैं।
Telugu listener hears: మీరు ఎలా ఉన్నారు? సరే, ఈరోజు మీటింగ్ మొదలుపెడదాం.
English listener hears: "How are you? Let's start today's meeting."

What it can do

- **All six directions:** Hindi ↔ English, Telugu ↔ English, Hindi ↔ Telugu.
- **Three AI translators** to choose from when creating a meeting: Google's Gemini 3.5 Live Translate (the default), OpenAI's GPT-Realtime-2, and OpenAI's GPT-Realtime-Translate.
- **Live captions** for both sides, with timing under each line (how fast the translation followed).
- **Works across networks**, including home Wi-Fi that blocks direct calls (through a relay server).
- **Safe to share:** creating a meeting needs an access code, so strangers can't run up the AI bill; joining needs only the invite link.

The approach in one sentence

Instead of building our own speech-recognition, translation and text-to-speech pipeline, we let **one AI model hear the speaker and speak the translation directly** (speech in, speech out), and spent our effort on everything around it: the call, timing, audio, safety and real-world bugs.

Best result

With Gemini, the translated voice runs **under ~2 seconds behind** the speaker and starts **while they are still talking**.

Biggest lesson

A model that works in a demo still needs a lot of work to behave in a **real call**: noise, pauses, echo, slow networks and misleading docs.

2. System design

The system has **five parts**. Only two of them are code we wrote: the **website** (frontend) and the **server** (backend). The rest are services we use.

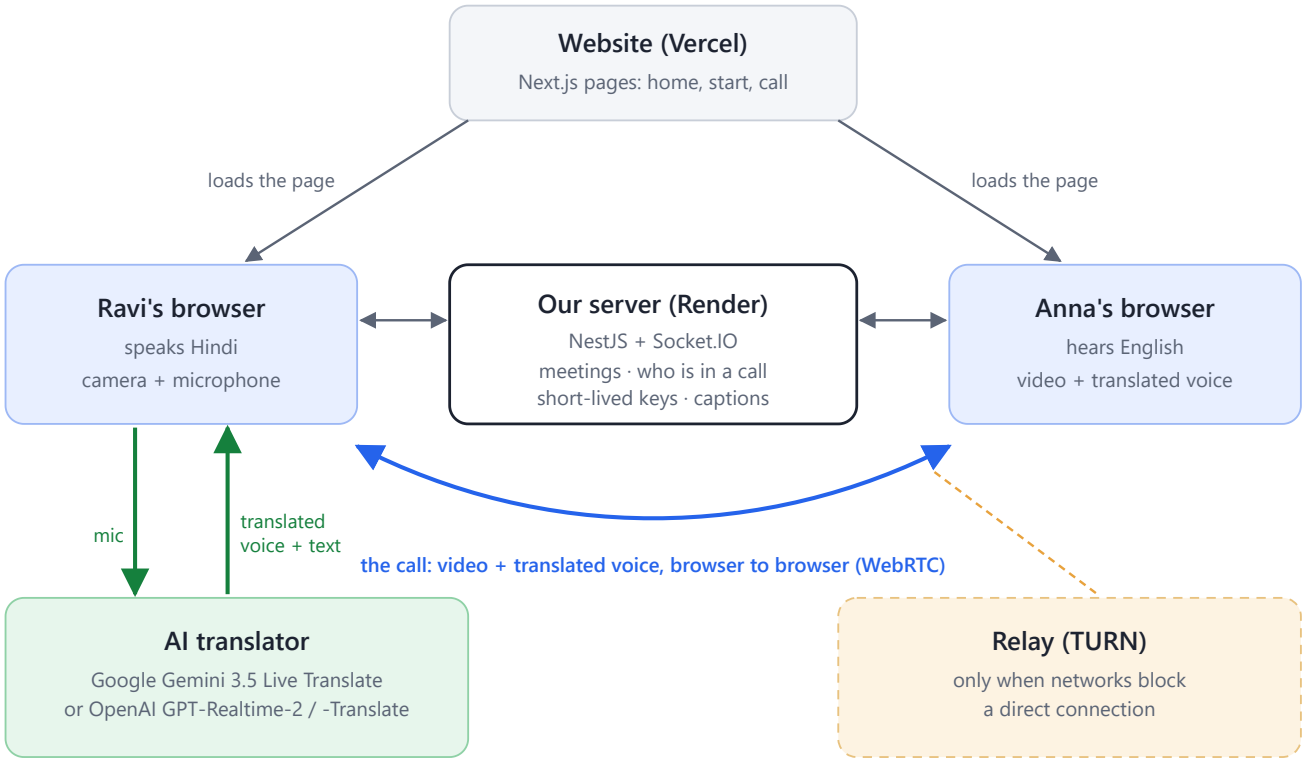


Figure 1 — The big picture. Grey arrows are small messages; the blue and green ones carry audio.

What each part does

Part	Its job, in plain words
Website (Vercel)	The pages people see: the home page, the "start or join a call" page, and the call itself. Almost all the real work (camera, microphone, audio) happens here, in the browser.
Our server (Render)	A small organiser. It creates meetings, remembers who is in each one, introduces the two browsers to each other, passes captions along, and hands out **short-lived keys** for the AI. It never touches the audio.
The call (WebRTC)	Video and voice go **directly** from one browser to the other — not through our server. This keeps it fast and cheap.
AI translator	Hears the speaker's microphone and sends back the translated voice and text. Google or OpenAI, chosen per meeting.
Relay (TURN)	Some home and office networks block direct connections. Then the call goes through this relay instead. Free tier from ExpressTURN.

One sentence, start to finish



The caption text takes a side path: Ravi's browser → our server → Anna's browser.
With Gemini, step 3 starts while Ravi is still talking; with GPT-Realtime-2, after he pauses.

Figure 2 — The life of one sentence.

How the AI key stays secret

The AI is paid per minute, so the real API key must never reach a browser (anyone could copy it from the page). So the browser gets a **short-lived, one-use key** instead:

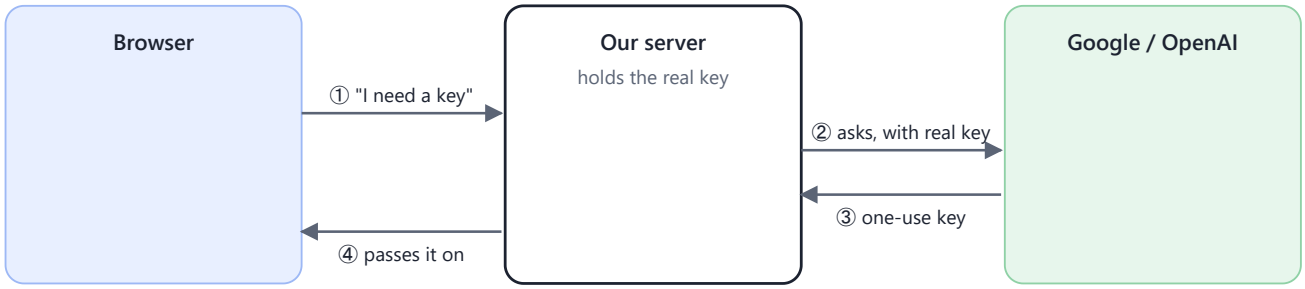


Figure 3 — The browser then talks to the AI directly with that key. It expires quickly and works for only one session.

The server only gives a key to someone who is **really in that meeting**, and only for a language pair the meeting's model can translate. For Gemini, the key is even **locked** to the right target language.

The four big design decisions

Decision	Why
Each speaker translates their own voice	The speaker's own microphone is the cleanest audio. Our first version translated on the listener's side, after the voice had crossed the (sometimes slow, relayed) call — real calls were noticeably slower.
Audio never passes through our server	Faster, cheaper, and the free server can stay tiny. The server only passes small messages.
No database	Meetings live in the server's memory. Simple on purpose: no saved transcripts, nothing personal stored. The trade-off: a server restart ends open meetings.
Model chosen per meeting	Lets us compare three models on real calls instead of guessing — which is how we found out Gemini works best.

3. The AI models

The models we used, in order

We tried the models one after another, each time because the previous one fell short on real calls. These are the exact model names in the code.

#	Added	Model (company)	What it does in our app	What happened
1	12 Oct	`gpt-realtime-translate` OpenAI	Our **first translator**: OpenAI's dedicated interpreter. Hindi/Telugu → English while the speaker talks. Takes only a target language, no instructions.	8 of 13 Hindi lines right; guessed early and corrected itself out loud; weak on Telugu; can't speak Telugu.
+	12 Oct	`gpt-realtime-whisper` OpenAI	A **helper, not a translator**: turns the speaker's own voice into text, shown under the translation. Used with both OpenAI models.	A Telugu word came out in Hindi script — fixed by telling it the speaker's language.
2	13 Oct	`gpt-realtime-2` OpenAI	OpenAI's **voice-assistant** model, turned into an interpreter by our prompt. Translates after each pause. Settings: minimal "thinking" (faster), a male voice ("cedar") at 1.2× speed.	17 of 18 real-call lines right. Added Hindi ↔ Telugu (13 Oct) and English → Hindi/Telugu (14 Oct).
3	14 Oct	`gemini-3.5-live-translate-preview` Google	Google's **dedicated interpreter**. Translates while the speaker talks, speaks Telugu too, and makes its own transcripts. The target language is locked into its one-use key.	All six directions worked on real calls, under ~2 s behind the speaker. **Now the default.**

Looked at but not used yet: OpenAI's newer GPT-Realtime-2.1 and 2.1-mini, and Google's Gemini 3.8 Live (an assistant model, like GPT-Realtime-2).

The three translators side by side

All three are "speech-to-speech": they hear audio and speak audio, with no separate text step that we have to manage. They differ in **when** they translate and **what they can do**.

	Gemini 3.5 Live Translate default	GPT-Realtime-2	GPT-Realtime-Translate
Made by	Google	OpenAI	OpenAI
When it translates	While you talk	After you pause	While you talk
Can speak Telugu	yes	yes	no
Directions	all 6	all 6	2 (into English only)
Follows our instructions	no	yes (our "interpreter" prompt)	no
Speed	under ~2 s behind the speaker	voice ~1.4 s after you stop	while you talk
Cost	~₹3 / minute	₹0.3–0.5 / sentence	~₹3 / minute
Connection	WebSocket (raw audio)	WebRTC	WebRTC

How we picked a winner

1. **We started with GPT-Realtime-Translate**, OpenAI's dedicated interpreter. On 13 real Hindi sentences it got 8 right, 2 partly right and 3 wrong — and real Telugu was much worse. Hindi puts the verb at the end, so it **guessed early and corrected itself out loud** ("Yes, the issue is fixed" … "Is the issue fixed?"). It can't take instructions, so we couldn't fix this, and it can't speak Telugu at all.
2. **We added GPT-Realtime-2**, OpenAI's voice-assistant model, and turned it into an interpreter with our own prompt ("never answer, only translate; keep numbers exact; translate idioms by meaning"). It waits for each sentence to end, so it gets the meaning right: **17 of 18 lines right** in real calls. The price is a delay after every sentence.
3. **We added Google's Gemini 3.5 Live Translate.** It translates while you talk (what we wanted), speaks Telugu, and covered all six directions in real calls. It became the default.

Why not just keep one model?

Keeping all three makes the comparison live and repeatable. It's also the honest story of the project: we didn't pick the "best" model from a blog post — we **measured** each one on real calls and let the results decide.

Same sentence, two OpenAI models

Speaker said (Hindi)	GPT-Realtime-Translate	GPT-Realtime-2 + our prompt
Kya aapne client se baat ki?	"Please speak with the client." wrong	"Did you talk to the client?" right
Ye kaam toh baayein haath ka khel hai	"You can do this work manually." wrong	"This work is a piece of cake." right
Budget do lakh pachaas hazaar	"two lakh", then "250,000" — both spoken	"two lakh fifty thousand rupees" right

4. How we built it

Three days, 23 small commits. We built the simplest working version first, tested it with **real people on real calls**, and fixed what broke.

When	What happened
Day 1 12 Oct	First version: a video call where a Hindi/Telugu speaker is heard in English, using GPT-Realtime-Translate. Real-voice test: works, but guesses out loud and struggles with Telugu.
Day 2 13 Oct	Added **GPT-Realtime-2** with an interpreter prompt, plus **Hindi ↔ Telugu**. **Deployed**: website on Vercel, server on Render, an access code, and a TURN relay for friends on other networks. A run of real-call fixes: translations cut off, quiet voices ignored, sentences invented from noise, slow calls → translation moved to the **speaker's side**.
Day 3 14 Oct	New **home page** with measured numbers, and a separate start page. **English → Hindi / Telugu**, so all six directions work. Added **Gemini 3.5 Live Translate**, tested it, made it the **default**. Renamed the project (it's no longer OpenAI-only) and moved the site to **meet-translate.vercel.app**. Every direction tested live.

How we tested

- **Automatic tests** (32) for the server: who may translate into what, and the exact requests sent to OpenAI and Google — against a fake server, so tests cost nothing.
- **Recorded voice clips** (Hindi and Telugu, generated with a text-to-speech tool) played as a fake microphone in two browser tabs, so a whole call could be tested without a second person.
- **Real calls** with a friend on another network, each followed by notes: what was said, what came out, how long it took. Most of the bugs in section 5 were only found this way.
- **Server log**: every finished line is logged with the original, the translation, its cost and its timing — no transcripts are stored.

Working style

Small steps. Each change was one commit with a plain message ("Stop made-up sentences from noise"), easy to explain and easy to undo.

Comments explain "why". Every fix in the code has a comment saying which real problem it solves, so the reasoning isn't lost.

5. Problems we faced, and how we solved them

Grouped by kind. "Cause" is what was really going on; "Fix" is what we changed.

A. Translation quality

The first model guessed early and corrected itself out loud

Cause	Hindi and Telugu put the verb (and "kya") at the end, so the meaning arrives last. A model that translates mid-sentence has to guess.
Fix	Added GPT-Realtime-2, which waits for the sentence to end, and later Gemini, where we haven't seen it so far.

Telugu "send the report" came out as "I will send the report"

Cause	Our prompt said "speak in the first person", but Telugu shows "you" only in the verb ending.
Fix	A new rule in the prompt: keep who does what exactly as said.

Questions got answered instead of translated

Cause	GPT-Realtime-2 is a voice assistant at heart: hearing "Did you talk to the client?" it replied "Yeah, I spoke with the client".
Fix	"Never answer" goes first and firmest in the prompt, plus a short reminder after every new speech: a question stays a question.

After a "hmm", it repeated the previous translation

Cause	Each reply saw the whole call; with nothing new to say, it said the last thing again.
Fix	Each reply sees only the new speech (plus the last few lines as "never repeat these"); filler sounds aren't sent at all.

Made-up sentences from breathing or background noise

Cause	Given a short noise, the model often invents a plausible sentence.
Fix	Short sounds are only translated if the speech-to-text finds real words in them.

B. Timing and audio

The next sentence cut off the translation still playing

Cause	OpenAI's own pause detection clears the playing audio when the speaker starts again — even with "don't interrupt" switched on.
Fix	We detect pauses ourselves in the browser (0.5 s of quiet, measured against the speaker's background noise) and play translations one at a time, each fully heard.

Nothing was translated for some speakers

Cause	A soft voice stayed under our loudness threshold; on a noisy line, the noise counted as endless speech, so no pause was ever found.
Fix	A lower threshold, and background noise learned all the time instead of only during silence.

Real calls felt slow, and the delay kept growing

Cause	The listener's browser did the translating, so the voice first crossed the (often relayed) call before reaching the AI; and translations queued up behind each other.
Fix	Moved translation to the speaker's side, on their clean mic; the translated voice speaks 1.2× faster so the queue catches up.

C. Connections and deployment

"Can't connect on this network" between two homes

Cause	Many home routers block direct browser-to-browser connections.
Fix	A TURN relay (ExpressTURN's free tier) that the call falls back to.

Everything went quiet when the free server restarted

Cause	Render's free plan sleeps after 15 idle minutes, and meetings live in memory.
Fix	Clear messages ("Reconnecting…", "the server restarted, create a new meeting"), and the home page starts waking the server while you read.

The website couldn't reach the server after deploying

Cause	A stray ">" from a placeholder had been pasted into a Vercel setting.
Fix	Corrected the value and redeployed; lesson: never use <angle-bracket> placeholders in setting values.

Google rejected our Gemini key request — although we followed its docs

Cause	The docs call the settings field `liveConnectConstraints`, but that's only the name inside Google's own code library; the actual API wants `bidiGenerateContentSetup`.
Fix	Read the source code of Google's library to see what it really sends, and sent the same.

"GEMINI_API_KEY is missing" — although a key had been added

Cause	Locally the key was saved as `GOOGLE_GEMINI_API_KEY`; on Render it wasn't set yet. The code looks for one exact name.
Fix	Used the exact name `GEMINI_API_KEY` in both places.

The new address meet-translate.vercel.app showed "Cannot reach the server"

Cause	For safety, the server only accepts requests from addresses listed in `FRONTEND_URL` (this is called CORS). The new address wasn't listed yet, and a trailing "/" or a missing restart also stops it matching.
Fix	Listed both addresses (old and new), exactly, and restarted the server. We also fixed a small bug found on the way: the live-call part didn't ignore spaces after the comma, while the rest did.

D. Still open

Gemini translates everything the microphone hears

Cause	It can't tell voices apart: in one test, a video playing in the background was translated too.
For now	Use headphones and close other audio. Not fixable on our side without a voice filter.

Sound leaking between apps on one laptop

Cause	Testing with two browsers (or with Discord on speakers) on one laptop lets each microphone hear the other app.
For now	Headphones for every test.

6. Results, tech stack, limits

Measured results

Gemini 3.5 Live Translate default

- Voice **under ~2 s behind** the speaker, in a real call
- Starts **~3.4 s after the speaker starts**, before a 3.9 s sentence had ended (recorded clip)
- **All 6 directions** worked on the live site
- About **₹3 a minute** (Google's price, silence included)

GPT-Realtime-2

- Caption **~1 s**, voice **~1.4 s** after the speaker stops
- **17 of 18 lines right** in real calls (Hindi → English 10/11, Telugu → Hindi 7/7)
- About **₹0.3–0.5 per sentence**; silence is nearly free

Tech stack

Layer	What we used	Why
Website	Next.js 16, React 19, TypeScript, Tailwind CSS	Standard, fast, free hosting on Vercel
Server	NestJS 11, Socket.IO, TypeScript	Clear structure; real-time messages
The call	WebRTC + TURN relay	Browser-to-browser, low delay
Translation	Gemini Live API (WebSocket), OpenAI Realtime API (WebRTC)	Speech in, speech out
Hosting	Vercel (website), Render (server)	Free tiers, deploy straight from GitHub
Tests	Jest — 32 tests, fake OpenAI/Google servers	Check real requests, spend nothing

Size: about 860 lines of server code, 1,860 lines of website code, 430 lines of tests.

Limits

- One-to-one calls only; no saved transcripts (no database, on purpose).
- Only Hindi, Telugu and English: OpenAI's models don't accept Tamil or Kannada.
- Gemini's maximum session length isn't documented; a very long call may need translation switched off and on.
- Meetings end if the free server restarts.

What's next

- A long call with Gemini, to find Google's session limit, and an accuracy count per direction.

- Try newer, cheaper OpenAI models (GPT-Realtime-2.1 and 2.1-mini).
- A TURN relay closer to India, to cut delay when the relay is used.

7. Glossary

The technical words in this report, in plain language.

Word	Meaning
Frontend / backend	The frontend is the website in your browser; the backend is our server that the website talks to.
WebRTC	The browser technology for video calls. It sends audio and video directly between two browsers.
WebSocket	An always-open connection for sending messages back and forth quickly. Gemini receives our audio this way.
Socket.IO	A library that makes WebSockets easy. Our server uses it for "someone joined", call setup messages, and captions.
TURN relay	A middle-man server that carries the call when two networks can't connect directly.
API key	A secret password that lets a program use a paid service like OpenAI or Google. Ours stays on the server.
Short-lived key / token	A temporary key that works once, for a few minutes. Safe to hand to a browser.
CORS	A browser safety rule: a website may only call a server that has listed that website's address as allowed.
Prompt	Written instructions given to an AI model. Ours turns a voice assistant into an interpreter.
Speech-to-speech model	An AI model that hears speech and answers with speech, without separate transcribe-translate-speak steps.
Latency	The delay: how long after someone speaks the other person hears the translation.
Environment variable	A setting given to the server from outside the code, like `GEMINI_API_KEY` or `FRONTEND_URL`, so secrets never go into the code.
Deploy	Putting a new version of the code online. Vercel does it on every push to GitHub; on Render we sometimes started it by hand.
Commit / push	A commit saves one change with a message; pushing uploads it to GitHub.

In one line: Meet Translate lets people who don't share a language talk on a video call, each hearing the other in their own language — built in three days by testing three AI models on real calls and fixing what real calls revealed.